

Chapter 8

Bayesian Learning

"In science, progress is possible. In fact, if one believes in Bayes' theorem, scientific progress is inevitable as predictions are made and as beliefs are tested and refined."

— Nate Silver

Bayesian Learning is a learning method that describes and represents knowledge in an uncertain domain and provides a way to reason about this knowledge using probability measure. It uses Bayes theorem to infer the unknown parameters of a model. Bayesian inference is useful in many applications which involve reasoning and diagnosis such as game theory, medicine, etc. Bayesian inference is much more powerful in handling missing data and for estimating any uncertainty in predictions.

Learning Objectives

- Understand the basics of probability-based learning and probability theory
- Learn the fundamentals of Bayes theorem
- Introduce Bayes Classification models such as Brute Force Bayes learning algorithm, Bayes Optimal classifier, and Gibbs algorithm
- Introduce Naïve Bayes Classification models that work on the principle of Bayes theorem
- Explore the Naïve Bayes classification algorithm
- Study about Naïve Bayes Algorithm for continuous attributes using Gaussian distribution
- Introduce other popular types of Naive Bayes classifiers such as Bernoulli Naive Bayes classifier, Multinomial Naive Bayes classifier, and Multi-class Naïve Bayes classifier

8.1 INTRODUCTION TO PROBABILITY-BASED LEARNING

Probability-based learning is one of the most important practical learning methods which combines prior knowledge or prior probabilities with observed data. Probabilistic learning uses the concept of probability theory that describes how to model randomness, uncertainty, and noise to predict future events. It is a tool for modelling large datasets and uses Bayes rule to infer unknown quantities, predict and learn from data. In a *probabilistic model*, randomness plays a major role which gives probability distribution a solution, while in a *deterministic model* there is no randomness and

hence it exhibits the same initial conditions every time the model is run and is likely to get a single possible outcome as the solution.

Bayesian learning differs from probabilistic learning as it uses subjective probabilities (i.e., probability that is based on an individual's belief or interpretation about the outcome of an event and it can change over time) to infer parameters of a model. Two practical learning algorithms called Naïve Bayes learning and Bayesian Belief Network (BBN) form the major part of Bayesian learning. These algorithms use prior probabilities and apply Bayes rule to infer useful information. Bayesian Belief Networks (BBN) is explained in detail in Chapter 9.

Scan for information on 'Probability Theory' and for 'Additional Examples'



8.2 FUNDAMENTALS OF BAYES THEOREM

Naïve Bayes Model relies on Bayes theorem that works on the principle of three kinds of probabilities called prior probability, likelihood probability, and posterior probability.

Prior Probability

It is the general probability of an uncertain event before an observation is seen or some evidence is collected. It is the initial probability that is believed before any new information is collected.

Likelihood Probability

Likelihood probability is the relative probability of the observation occurring for each class or the sampling density for the evidence given the hypothesis. It is stated as $P(\text{Evidence} \mid \text{Hypothesis})$, which denotes the likeliness of the occurrence of the evidence given the parameters.

Posterior Probability

It is the updated or revised probability of an event taking into account the observations from the training data. $P(\text{Hypothesis} \mid \text{Evidence})$ is the posterior distribution representing the belief about the hypothesis, given the evidence from the training data. Therefore,

Posterior probability = prior probability + new evidence

8.3 CLASSIFICATION USING BAYES MODEL

Naïve Bayes Classification models work on the principle of Bayes theorem. Bayes' rule is a mathematical formula used to determine the posterior probability, given prior probabilities of events. Generally, Bayes theorem is used to select the most probable hypothesis from data, considering both prior knowledge and posterior distributions. It is based on the calculation of the posterior probability and is stated as:

$P(\text{Hypothesis } h \mid \text{Evidence } E)$

where, Hypothesis h is the target class to be classified and Evidence E is the given test instance.

$P(\text{Hypothesis } h | \text{Evidence } E)$ is calculated from the prior probability $P(\text{Hypothesis } h)$, the likelihood probability $P(\text{Evidence } E | \text{Hypothesis } h)$ and the marginal probability $P(\text{Evidence } E)$. It can be written as:

$$P(\text{Hypothesis } h | \text{Evidence } E) = \frac{P(\text{Evidence } E | \text{Hypothesis } h) P(\text{Hypothesis } h)}{P(\text{Evidence } E)} \quad (8.1)$$

where, $P(\text{Hypothesis } h)$ is the prior probability of the hypothesis h without observing the training data or considering any evidence. It denotes the prior belief or the initial probability that the hypothesis h is correct. $P(\text{Evidence } E)$ is the prior probability of the evidence E from the training dataset without any knowledge of which hypothesis holds. It is also called the marginal probability.

$P(\text{Evidence } E | \text{Hypothesis } h)$ is the prior probability of Evidence E given Hypothesis h . It is the likelihood probability of the Evidence E after observing the training data that the hypothesis h is correct. $P(\text{Hypothesis } h | \text{Evidence } E)$ is the posterior probability of Hypothesis h given Evidence E . It is the probability of the hypothesis h after observing the training data that the evidence E is correct. In other words, by the equation of Bayes Eq. (8.1), one can observe that:

Posterior Probability $\propto$ Prior Probability $\times$ Likelihood Probability

Bayes theorem helps in calculating the posterior probability for a number of hypotheses, from which the hypothesis with the highest probability can be selected.

This selection of the most probable hypothesis from a set of hypotheses is formally defined as Maximum A Posteriori (MAP) Hypothesis.

Maximum A Posteriori (MAP) Hypothesis, h_{MAP}

Given a set of candidate hypotheses, the hypothesis which has the maximum value is considered as the *maximum probable hypothesis* or *most probable hypothesis*. This most probable hypothesis is called the Maximum A Posteriori Hypothesis h_{MAP} . Bayes theorem Eq. (8.1) can be used to find the h_{MAP} .

$$\begin{aligned} h_{MAP} &= \max_{h \in H} P(\text{Hypothesis } h | \text{Evidence } E) \\ &= \max_{h \in H} \frac{P(\text{Evidence } E | \text{Hypothesis } h) P(\text{Hypothesis } h)}{P(\text{Evidence } E)} \\ &= \max_{h \in H} P(\text{Evidence } E | \text{Hypothesis } h) P(\text{Hypothesis } h) \end{aligned} \quad (8.2)$$

Maximum Likelihood (ML) Hypothesis, h_{ML}

Given a set of candidate hypotheses, if every hypothesis is equally probable, only $P(E | h)$ is used to find the *most probable hypothesis*. The hypothesis that gives the maximum likelihood for $P(E | h)$ is called the Maximum Likelihood (ML) Hypothesis, h_{ML} .

$$h_{ML} = \max_{h \in H} P(\text{Evidence } E | \text{Hypothesis } h) \quad (8.3)$$

Correctness of Bayes Theorem

Consider two events A and B in a sample space S.

A T F T T F T T F

B F T T F T F T F

$P(A) = 5/8$

$P(B) = 4/8$

$$P(A | B) = 2/4$$

$$P(B | A) = 2/5$$

$$P(A | B) = P(B | A) P(A) / P(B) = 2/4$$

$$P(B | A) = P(A | B) P(B) / P(A) = 2/5$$

Let us consider a numerical example to illustrate the use of Bayes theorem now:

Example 8.1: Consider a boy who has a volleyball tournament on the next day, but today he feels sick. It is unusual that there is only a 40% chance he would fall sick since he is a healthy boy. Now, Find the probability of the boy participating in the tournament. The boy is very much interested in volley ball, so there is a 90% probability that he would participate in tournaments and 20% that he will fall sick given that he participates in the tournament.

Solution: $P(\text{Boy participating in the tournament}) = 90\%$

$$P(\text{He is sick} | \text{Boy participating in the tournament}) = 20\%$$

$$P(\text{He is Sick}) = 40\%$$

The probability of the boy participating in the tournament given that he is sick is:

$$P(\text{Boy participating in the tournament} | \text{He is sick}) = P(\text{Boy participating in the tournament}) \times P(\text{He is sick} | \text{Boy participating in the tournament}) / P(\text{He is Sick})$$

$$P(\text{Boy participating in the tournament} | \text{He is sick}) = (0.9 \times 0.2) / 0.4 \\ = 0.45$$

Hence, 45% is the probability that the boy will participate in the tournament given that he is sick.

One related concept of Bayes theorem is the principle of Minimum Description Length (MDL). The minimum description length (MDL) principle is yet another powerful method like Occam's razor principle to perform inductive inference. It states that the best and most probable hypothesis is chosen for a set of observed data or the one with the minimum description. Recall from Eq. (8.2) Maximum A Posteriori (MAP) Hypothesis, h_{MAP} which says that given a set of candidate hypotheses, the hypothesis which has the maximum value is considered as the *maximum probable hypothesis* or *most probable hypothesis*. Naïve Bayes algorithm uses the Bayes theorem and applies this MDL principle to find the best hypothesis for a given problem. Let us clearly understand how this algorithm works in the following Section 8.3.1.

8.3.1 NAÏVE BAYES ALGORITHM

It is a supervised binary class or multi class classification algorithm that works on the principle of Bayes theorem. There is a family of Naïve Bayes classifiers based on a common principle. These algorithms classify for datasets whose features are independent and each feature is assumed to be given equal weightage. It particularly works for a large dataset and is very fast. It is one of the most effective and simple classification algorithms. This algorithm considers all features to be independent of each other even though they are individually dependent on the classified object. Each of the features contributes a probability value independently during classification and hence this algorithm is called as Naïve algorithm.

Some important applications of these algorithms are text classification, recommendation system and face recognition.

Algorithm 8.1: Naïve Bayes

1. Compute the prior probability for the target class.
2. Compute Frequency matrix and likelihood Probability for each of the feature.
3. Use Bayes theorem Eq. (8.1) to calculate the probability of all hypotheses.
4. Use Maximum A Posteriori (MAP) Hypothesis, h_{MAP} Eq. (8.2) to classify the test object to the hypothesis with the highest probability.

Example 8.2: Assess a student's performance using Naïve Bayes algorithm with the dataset provided in Table 8.1. Predict whether a student gets a job offer or not in his final year of the course.

Table 8.1: Training Dataset

S.No.	CGPA	Interactiveness	Practical Knowledge	Communication Skills	Job Offer
1.	≥ 9	Yes	Very good	Good	Yes
2.	≥ 8	No	Good	Moderate	Yes
3.	≥ 9	No	Average	Poor	No
4.	< 8	No	Average	Good	No
5.	≥ 8	Yes	Good	Moderate	Yes
6.	≥ 9	Yes	Good	Moderate	Yes
7.	< 8	Yes	Good	Poor	No
8.	≥ 9	No	Very good	Good	Yes
9.	≥ 8	Yes	Good	Good	Yes
10.	≥ 8	Yes	Average	Good	Yes

Solution: The training dataset T consists of 10 data instances with attributes such as 'CGPA', 'Interactiveness', 'Practical Knowledge' and 'Communication Skills' as shown in Table 8.1. The target variable is Job Offer which is classified as Yes or No for a candidate student.

Step 1: Compute the prior probability for the target feature 'Job Offer'. The target feature 'Job Offer' has two classes, 'Yes' and 'No'. It is a binary classification problem. Given a student instance, we need to classify whether 'Job Offer = Yes' or 'Job Offer = No'.

From the training dataset, we observe that the frequency or the number of instances with 'Job Offer = Yes' is 7 and 'Job Offer = No' is 3.

The prior probability for the target feature is calculated by dividing the number of instances belonging to a particular target class by the total number of instances.

Hence, the prior probability for 'Job Offer = Yes' is $7/10$ and 'Job Offer = No' is $3/10$ as shown in Table 8.2.

Table 8.2: Frequency Matrix and Prior Probability of Job Offer

Job Offer Classes	No. of Instances	Probability Value
Yes	7	$P(\text{Job Offer} = \text{Yes}) = 7/10$
No	3	$P(\text{Job Offer} = \text{No}) = 3/10$

Step 2: Compute Frequency matrix and Likelihood Probability for each of the feature.

Step 2(a): Feature – CGPA

Table 8.3 shows the frequency matrix for the feature CGPA.

Table 8.3: Frequency Matrix of CGPA

CGPA	Job Offer = Yes	Job Offer = No
≥ 9	3	1
≥ 8	4	0
< 8	0	2
Total	7	3

Table 8.4 shows how the likelihood probability is calculated for CGPA using conditional probability.

Table 8.4: Likelihood Probability of CGPA

CGPA	$P(\text{Job Offer} = \text{Yes})$	$P(\text{Job Offer} = \text{No})$
≥ 9	$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) = 3/7$	$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) = 1/3$
≥ 8	$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) = 4/7$	$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) = 0/3$
< 8	$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{Yes}) = 0/7$	$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{No}) = 2/3$

As explained earlier the Likelihood probability is stated as the sampling density for the evidence given the hypothesis. It is denoted as $P(\text{Evidence} \mid \text{Hypothesis})$, which says how likely is the occurrence of the evidence given the parameters.

It is calculated as the number of instances of each attribute value and for a given class value divided by the number of instances with that class value.

For example $P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes})$ denotes the number of instances with 'CGPA ≥ 9 ' and 'Job Offer = Yes' divided by the total number of instances with 'Job Offer = Yes'.

From the Table 8.3 Frequency Matrix of CGPA, number of instances with 'CGPA ≥ 9 ' and 'Job Offer = Yes' is 3. The total number of instances with 'Job Offer = Yes' is 7. Hence, $P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) = 3/7$.

Similarly, the Likelihood probability is calculated for all attribute values of feature CGPA.

Step 2(b): Feature – Interactiveness

Table 8.5 shows the frequency matrix for the feature Interactiveness.

Table 8.5: Frequency Matrix of Interactiveness

Interactiveness	Job Offer = Yes	Job Offer = No
YES	5	1
NO	2	2
Total	7	3

Table 8.6 shows how the likelihood probability is calculated for Interactiveness using conditional probability.

Table 8.6: Likelihood Probability of Interactiveness

Interactiveness	P (Job Offer = Yes)	P (Job Offer = No)
YES	$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) = 5/7$	$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) = 1/3$
NO	$P(\text{Interactiveness} = \text{No} \mid \text{Job Offer} = \text{Yes}) = 2/7$	$P(\text{Interactiveness} = \text{No} \mid \text{Job Offer} = \text{No}) = 2/3$

Step 2(c): Feature – Practical Knowledge

Table 8.7 shows the frequency matrix for the feature Practical Knowledge.

Table 8.7: Frequency Matrix of Practical Knowledge

Practical Knowledge	Job Offer = Yes	Job Offer = No
Very Good	2	0
Average	1	2
Good	4	1
Total	7	3

Table 8.8 shows how the likelihood probability is calculated for Practical Knowledge using conditional probability.

Table 8.8: Likelihood Probability of Practical Knowledge

Practical Knowledge	P (Job Offer = Yes)	P (Job Offer = No)
Very Good	$P(\text{Practical Knowledge} = \text{Very Good} \mid \text{Job Offer} = \text{Yes}) = 2/7$	$P(\text{Practical Knowledge} = \text{Very Good} \mid \text{Job Offer} = \text{No}) = 0/3$
Average	$P(\text{Practical Knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) = 1/7$	$P(\text{Practical Knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) = 2/3$
Good	$P(\text{Practical Knowledge} = \text{Good} \mid \text{Job Offer} = \text{Yes}) = 4/7$	$P(\text{Practical Knowledge} = \text{Good} \mid \text{Job Offer} = \text{No}) = 1/3$

Step 2(d): Feature – Communication Skills

Table 8.9 shows the frequency matrix for the feature Communication Skills.

Table 8.9: Frequency Matrix of Communication Skills

Communication Skills	Job Offer = Yes	Job Offer = No
Good	4	1
Moderate	3	0
Poor	0	2
Total	7	3

Table 8.10 shows how the likelihood probability is calculated for Communication Skills using conditional probability.

Table 8.10: Likelihood Probability of Communication Skills

Communication Skills	$P(\text{Job Offer} = \text{Yes})$	$P(\text{Job Offer} = \text{No})$
Good	$P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) = 4/7$	$P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) = 1/3$
Moderate	$P(\text{Communication Skills} = \text{Moderate} \mid \text{Job Offer} = \text{Yes}) = 3/7$	$P(\text{Communication Skills} = \text{Moderate} \mid \text{Job Offer} = \text{No}) = 0/3$
Poor	$P(\text{Communication Skills} = \text{Poor} \mid \text{Job Offer} = \text{Yes}) = 0/7$	$P(\text{Communication Skills} = \text{Poor} \mid \text{Job Offer} = \text{No}) = 2/3$

Step 3: Use Bayes theorem Eq. (8.1) to calculate the probability of all hypotheses.

Given the test data = (CGPA ≥ 9 , Interactiveness = Yes, Practical knowledge = Average, Communication Skills = Good), apply the Bayes theorem to classify whether the given student gets a Job offer or not.

$$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes})) / (P(\text{Test Data}))$$

We can ignore $P(\text{Test Data})$ in the denominator since it is common for all cases to be considered.

$$\begin{aligned} \text{Hence, } P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) &= (P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes})) \\ &= 3/7 \times 5/7 \times 1/7 \times 4/7 \times 7/10 \\ &= 0.0175 \end{aligned}$$

$$= 0.0175$$

Similarly, for the other case 'Job Offer = No',

We compute the probability,

$$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = (P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No})) / (P(\text{Test Data})).$$

$$\begin{aligned} P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No}) \\ = 1/3 \times 1/3 \times 2/3 \times 1/3 \times 3/10 \\ = 0.0074 \end{aligned}$$

Step 4: Use Maximum A Posteriori (MAP) Hypothesis, h_{MAP} Eq. (8.2) to classify the test object to the hypothesis with the highest probability.

Since $P(\text{Job Offer} = \text{Yes} \mid \text{Test data})$ has the highest probability value, the test data is classified as 'Job Offer = Yes'.

Zero Probability Error

In Example 8.1, consider the test data to be (CGPA ≥ 8 , Interactiveness = Yes, Practical knowledge = Average, Communication Skills = Good)

When computing the posterior probability,

$$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes})) / (P(\text{Test Data}))$$

$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes}))$

$$= 4/7 \times 5/7 \times 1/7 \times 4/7 \times 7/10$$

$$= 0.0233$$

Similarly, for the other case 'Job Offer = No',

When we compute the probability:

$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = (P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No})) / (P(\text{Test Data}))$

$= P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No})$

$$= 0/3 \times 1/3 \times 2/3 \times 1/3 \times 3/10$$

$$= 0$$

Since the probability value is zero, the model fails to predict, and this is called as Zero-Probability error. This problem arises because there are no instances in the given Table 8.1 for the attribute value $\text{CGPA} \geq 8$ and $\text{Job Offer} = \text{No}$ and hence the probability value of this case is zero. This zero-probability error can be solved by applying a smoothing technique called Laplace correction which means given 1000 data instances in the training dataset, if there are zero instances for a particular value of a feature we can add 1 instance for each attribute value pair of that feature which will not make much difference for 1000 data instances and the overall probability does not become zero.

Now, let us scale the values given in Table 8.1 for 1000 data instances. The scaled values without Laplace correction are shown in Table 8.11.

Table 8.11: Scaled Values to 1000 without Laplace Correction

CGPA	$P(\text{Job Offer} = \text{Yes})$	$P(\text{Job Offer} = \text{No})$
≥ 9	$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) = 300/700$	$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) = 100/300$
≥ 8	$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) = 400/700$	$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) = 0/300$
< 8	$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{Yes}) = 0/700$	$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{No}) = 200/300$

Now, add 1 instance for each CGPA-value pair for 'Job Offer = No'. Then,

$$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) = 101/303 = 0.333$$

$$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) = 1/303 = 0.0033$$

$$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{No}) = 201/303 = 0.6634$$

With scaled values to 1003 data instances, we get

$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes}))$

$$= 400/700 \times 500/700 \times 100/700 \times 400/700 \times 700/1003$$

$$= 0.02325$$

$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No})$

$$= 1/303 \times 100/300 \times 200/300 \times 100/300 \times 303/1003$$

$$= 0.00007385$$

Thus, using Laplace Correction, Zero Probability error can be solved with Naïve Bayes classifier.

8.3.2 Brute Force Bayes Algorithm

Applying Bayes theorem, Brute Force Bayes algorithm relies on the idea of concept learning wherein given a hypothesis space H for the training dataset T , the algorithm computes the posterior probabilities for all the hypothesis $h_i \in H$. Then, Maximum A Posteriori (MAP) Hypothesis, h_{MAP} is used to output the hypothesis with maximum posterior probability. The algorithm is quite expensive since it requires computations for all the hypotheses. Although computing posterior probabilities is inefficient, this idea is applied in various other algorithms which is also quite interesting.

8.3.3 Bayes Optimal Classifier

Bayes optimal classifier is a probabilistic model, which in fact, uses the Bayes theorem to find the most probable classification for a new instance given the training data by combining the predictions of all posterior hypotheses. This is different from Maximum A Posteriori (MAP) Hypothesis, h_{MAP} which chooses the maximum probable hypothesis or the most probable hypothesis.

Here, a new instance can be classified to a possible classification value C_i by the following Eq. (8.4).

$$= \max_{C_i} \sum_{h_i \in H} P(C_i \mid h_i) P(h_i \mid T) \quad (8.4)$$

Example 8.3: Given the hypothesis space with 4 hypothesis h_1 , h_2 , h_3 and h_4 . Determine if the patient is diagnosed as COVID positive or COVID negative using Bayes Optimal classifier.

Solution: From the training dataset T , the posterior probabilities of the four different hypotheses for a new instance are given in Table 8.12.

Table 8.12: Posterior Probability Values

$P(h_i \mid T)$	$P(\text{COVID Positive} \mid h_i)$	$P(\text{COVID Negative} \mid h_i)$
0.3	0	1
0.1	1	0
0.2	1	0
0.1	1	0

h_{MAP} chooses h_1 which has the maximum probability value 0.3 as the solution and gives the result that the patient is COVID negative. But Bayes Optimal classifier combines the predictions of h_2 , h_3 and h_4 which is 0.4 and gives the result that the patient is COVID positive.

$$\sum_{h_i \in H} P(\text{COVID Negative} \mid h_i) P(h_i \mid T) = 0.3 \times 1 = 0.3$$

$$\sum_{h_i \in H} P(\text{COVID Positive} \mid h_i) P(h_i \mid T) = 0.1 \times 1 + 0.2 \times 1 + 0.1 \times 1 = 0.4$$

Therefore, $\max_{C_i \in \{\text{COVID Positive}, \text{COVID Negative}\}} \sum_{h_i \in H} P(C_i | h_i) P(h_i | T) = \text{COVID Positive}$.

Thus, this algorithm, diagnoses the new instance to be COVID positive.

8.3.4 Gibbs Algorithm

The main drawback of Bayes optimal classifier is that it computes the posterior probability for all hypotheses in the hypothesis space and then combines the predictions to classify a new instance.

Gibbs algorithm is a sampling technique which randomly selects a hypothesis from the hypothesis space according to the posterior probability distribution and classifies a new instance. It is found that the prediction error occurs twice with the Gibbs algorithm when compared to Bayes Optimal classifier.

8.4 NAÏVE BAYES ALGORITHM FOR CONTINUOUS ATTRIBUTES

There are two ways to predict with Naive Bayes algorithm for continuous attributes:

1. Discretize continuous feature to discrete feature.
2. Apply Normal or Gaussian distribution for continuous feature.

Gaussian Naive Bayes Algorithm

In Gaussian Naive Bayes, the values of continuous features are assumed to be sampled from a Gaussian distribution.

Example 8.4: Assess a student's performance using Naïve Bayes algorithm for the continuous attribute. Predict whether a student gets a job offer or not in his final year of the course. The training dataset T consists of 10 data instances with attributes such as 'CGPA' and 'Interactiveness' as shown in Table 8.13. The target variable is Job Offer which is classified as Yes or No for a candidate student.

Table 8.13: Training Dataset with Continuous Attribute

S.No.	CGPA	Interactiveness	Job Offer
1.	9.5	Yes	Yes
2.	8.2	No	Yes
3.	9.3	No	No
4.	7.6	No	No
5.	8.4	Yes	Yes
6.	9.1	Yes	Yes
7.	7.5	Yes	No
8.	9.6	No	Yes
9.	8.6	Yes	Yes
10.	8.3	Yes	Yes

Solution:

Step 1: Compute the prior probability for the target feature 'Job Offer'.

Prior probabilities of both the classes are calculated using the same formula (refer to Table 8.14).

Table 8.14: Prior Probability of Target Class

Job Offer Classes	No. of Instances	Probability Value
Yes	7	$P(\text{Job Offer} = \text{Yes}) = 7/10$
No	3	$P(\text{Job Offer} = \text{No}) = 3/10$

Step 2: Compute Frequency matrix and Likelihood Probability for each of the feature.

Likelihood probabilities for a continuous attribute is obtained from Gaussian (Normal) Distribution. In the above data set, CGPA is a continuous attribute for which we need to apply Gaussian distribution to calculate the likelihood probability.

Gaussian distribution for continuous feature is calculated using the given formula,

$$P(X_i = x_k | C_j) = g(x_k, \mu_{ij}, \sigma_{ij}) \quad (8.5)$$

where,

X_i is the i^{th} continuous attribute in the given dataset and x_k is a value of the attribute.

C_j denotes the j^{th} class of the target feature.

μ_{ij} denotes the mean of the values of that continuous attribute X_i with respect to the class j of the target feature.

σ_{ij} denotes the standard deviation of the values of that continuous attribute X_i with respect to the class j of the target feature.

Hence, the normal distribution formula is given as:

$$P(X_i = x_k | C_j) = \frac{1}{\sigma_{ij} \sqrt{2\pi}} e^{-\frac{(x_k - \mu_{ij})^2}{2\sigma_{ij}^2}} \quad (8.6)$$

Step 2(a): Consider the feature CGPA

In this example CGPA is a continuous attribute,

To calculate the likelihood probability for this continuous attribute, first compute the mean and standard deviation for CGPA with respect to the target class 'Job Offer'.

Here, $X_i = \text{CGPA}$

$C_j = \text{'Job Offer' = Yes'}$

Mean and Standard Deviation for class 'Job Offer = Yes' are given as:

$$\mu_{ij} = \mu_{\text{CGPA} - \text{YES}} = 8.814286$$

$$\sigma_{ij} = \sigma_{\text{CGPA} - \text{YES}} = 0.58146$$

Mean and Standard Deviation for class 'Job Offer = No' are given as:

$C_j = \text{'Job Offer' = No'}$

$$\mu_{ij} = \mu_{\text{CGPA} - \text{NO}} = 8.133333$$

$$\sigma_{ij} = \sigma_{\text{CGPA} - \text{NO}} = 1.011599$$

Once Mean and Standard Deviation are computed, the likelihood probability for any test value using Gaussian distribution formula can be calculated.

Step 2(b): Consider the feature Interactiveness

Interactiveness is a discrete feature whose probability is calculated as earlier.

Table 8.15 shows the frequency matrix for the feature Interactiveness.

Table 8.15: Frequency Matrix of Interactiveness

Interactiveness	Job Offer = Yes	Job Offer = No
YES	5	1
NO	2	2
Total	7	3

Table 8.16 shows how the likelihood probability is calculated for Interactiveness using conditional probability.

Table 8.16: Likelihood Probability of Interactiveness

Interactiveness	$P(\text{Job Offer} = \text{Yes})$	$P(\text{Job Offer} = \text{No})$
YES	$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) = 5/7$	$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) = 1/3$
NO	$P(\text{Interactiveness} = \text{No} \mid \text{Job Offer} = \text{Yes}) = 2/7$	$P(\text{Interactiveness} = \text{No} \mid \text{Job Offer} = \text{No}) = 2/3$

Step 3: Use Bayes theorem to calculate the probability of all hypotheses.

Consider the test data to be (CGPA = 8.5, Interactiveness = Yes).

For the hypothesis 'Job Offer = Yes':

$$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{Yes}) \times P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) \times P(\text{Job Offer} = \text{Yes}))$$

To compute $P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{Yes})$ use Gaussian distribution formula:

$$P(X_i = x_k \mid C_j) = g(x_k, \mu_{ij}, \sigma_{ij})$$

$$P(X_{\text{CGPA}} = 8.5 \mid C_{\text{Job Offer} = \text{Yes}}) = \frac{1}{\sigma_{ij} \sqrt{2\pi}} e^{-\frac{(x_k - \mu_{ij})^2}{2\sigma_{ij}^2}}$$

$$P(X_{\text{CGPA}} = 8.5 \mid C_{\text{Job Offer} = \text{Yes}}) = \frac{1}{\sigma_{\text{CGPA-YES}} \sqrt{2\pi}} e^{-\frac{(8.5 - \mu_{\text{CGPA-YES}})^2}{2\sigma_{\text{CGPA-YES}}^2}}$$

$$P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{Yes}) = g(x_k = 8.5, \mu_{ij} = 8.814, \sigma_{ij} = 0.581)$$

$$= \frac{1}{0.581 \sqrt{2\pi}} e^{-\frac{(8.5 - 8.814)^2}{2 \times 0.581^2}} = 0.594$$

$$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) = 5/7$$

$$P(\text{Job Offer} = \text{Yes}) = 7/10$$

Hence:

$$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{Yes}) \times P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) \times P(\text{Job Offer} = \text{Yes}))$$

$$= 0.594 \times 5/7 \times 7/10$$

$$= 0.297$$

Similarly, for the hypothesis 'Job Offer = No':

$$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{No}) \times P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) \times P(\text{Job Offer} = \text{No})$$

$$P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{No}) = g(x_k = 8.5, \mu_{ij} = 8.133, \sigma_{ij} = 1.0116)$$

$$P(X_{\text{CGPA}} = 8.5 \mid C_{\text{Job Offer} = \text{Yes}}) = \frac{1}{\sigma_{\text{CGPA-NO}} \sqrt{2\pi}} e^{-\frac{(8.5 - \mu_{\text{CGPA-NO}})^2}{2\sigma_{\text{CGPA-NO}}^2}}$$

$$= \frac{1}{1.0116 \sqrt{2\pi}} e^{-\frac{(8.5 - 8.133)^2}{2 \times 1.0116^2}} = 0.369$$

$$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) = 1/3$$

$$= P(\text{Job Offer} = \text{No}) = 0.369$$

Hence,

$$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) \times P(\text{Job Offer} = \text{No})$$

$$= 0.369 \times 1/3 \times 3/10$$

$$= 0.0369$$

Step 4: Use Maximum A Posteriori (MAP) Hypothesis, h_{MAP} to classify the test object to the hypothesis with the highest probability.

Since $P(\text{Job Offer} = \text{Yes} \mid \text{Test data})$ has the highest probability value of 0.297, the test data is classified as 'Job Offer = Yes'.



Chapter 10

Artificial Neural Networks

"I think the brain is essentially a computer and consciousness is like a computer program. It will cease to run when the computer is turned off. Theoretically, it could be re-created on a neural network, but that would be very difficult, as it would require all of one's memories."

— Stephen Hawking

Artificial Neural Networks (ANNs) imitate human brain behaviour and the way in which learning happens in a human. The human brain constitutes a mass of neurons that are all connected as a network, which is actually a directed graph. These neurons are the processing units which receive information, process it and then transmit this data to other neurons that allows humans to learn almost any task. ANN is a learning mechanism that models a human brain to solve any non-linear and complex problem. Each neuron is modelled as a computing unit, or simply called as a node in ANN, that is capable of doing complex calculations. ANN is a system that consists of many such computing units operating in parallel that can learn from observations. They are widely used in developing artificial learning systems and have inspired researchers and industry in Machine Learning nowadays. Some typical applications of ANN in the field of computer science are Natural Language Processing (NLP), pattern recognition, face recognition, speech recognition, character recognition, text processing, stock prediction, computer vision, etc. ANNs also have been considerably used in other engineering fields such as Chemical industry, Medicine, Robotics, Communications, Banking, and Marketing. This chapter aims to introduce the concepts necessary to understand the working of Artificial Neural Networks.

Learning Objectives

- Understand the basics of human nervous system and biological neurons
- Learn about McCulloch & Pitts Neuron Mathematical Model of an artificial neuron
- Know about the structure of Artificial Neural Network
- Introduce different types of activation functions
- Explore about the first neural network model called 'Perceptron' which is a linear binary classifier used for supervised learning
- Introduce different types of Artificial Neural Networks
- Understand the concepts of learning in Multi-Layer Perceptron (MLP) using back propagation
- Study about Radial Basis Function Neural Network (RBFNN) which is a type of multi-layer perceptron particularly useful for interpolation, function approximation, time series prediction, classification and system control
- Explore Self Organizing Feature Map (SOFM) which is a competitive learning network for unsupervised learning

10.1 INTRODUCTION

The human nervous system has billions of neurons that are the processing units which make humans to perceive things, to hear, to see and to smell. The human nervous system works beautifully, making us understand who we are, what we do, where we are and everything in our surrounding. It makes us to remember, recognize and correlate things around us. It is a learning system that consists of functional units called nerve cells, typically called as neurons. The human nervous system is divided into two sections called the Central Nervous System (CNS) and the Peripheral Nervous System (PNS). The brain and the spinal cord constitute the CNS and the neurons inside and outside the CNS constitute the PNS. The neurons are basically classified into three types called sensory neurons, motor neurons and interneurons. Sensory neurons get information from different parts of the body and bring it into the CNS, whereas motor neurons receive information from other neurons and transmit commands to the body parts. The CNS consists of only interneurons which connect one neuron to another neuron by receiving information from one neuron and transmitting it to another. The basic functionality of a neuron is to receive information, process it and then transmit it to another neuron or to a body part.

Scan for information on 'Convolution Neural Network', 'Modular Neural Network', and 'Recurrent Neural Network'



10.2 BIOLOGICAL NEURONS

A typical biological neuron has four parts called dendrites, soma, axon and synapse. The body of the neuron is called as soma. Dendrites accept the input information and process it in the cell body called soma. A single neuron is connected by axons to around 10,000 neurons and through these axons the processed information is passed from one neuron to another neuron. A neuron gets fired if the input information crosses a threshold value and transmits signals to another neuron through a synapse. A synapse gets fired with an electrical impulse called spikes which are transmitted to another neuron. A single neuron can receive synaptic inputs from one neuron or multiple neurons. These neurons form a network structure which processes input information and gives out a response. The simple structure of a biological neuron is shown in Figure 10.1.

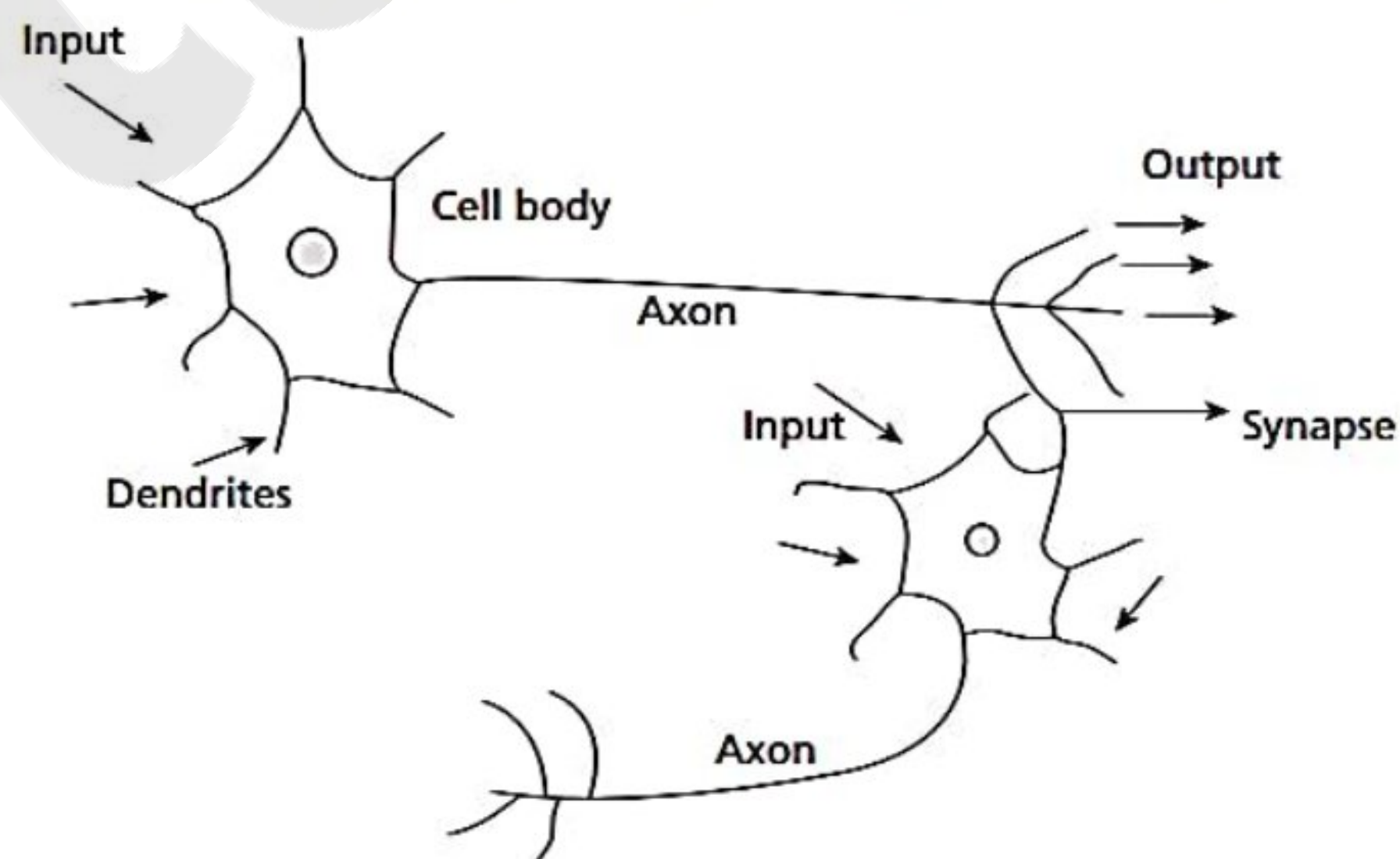


Figure 10.1: A Biological Neuron

10.3 ARTIFICIAL NEURONS

Artificial neurons are like biological neurons which are called as nodes. A node or a neuron can receive one or more input information and process it. Artificial neurons or nodes are connected by connection links to one another. Each connection link is associated with a synaptic weight. The structure of a single neuron is shown in Figure 10.2.

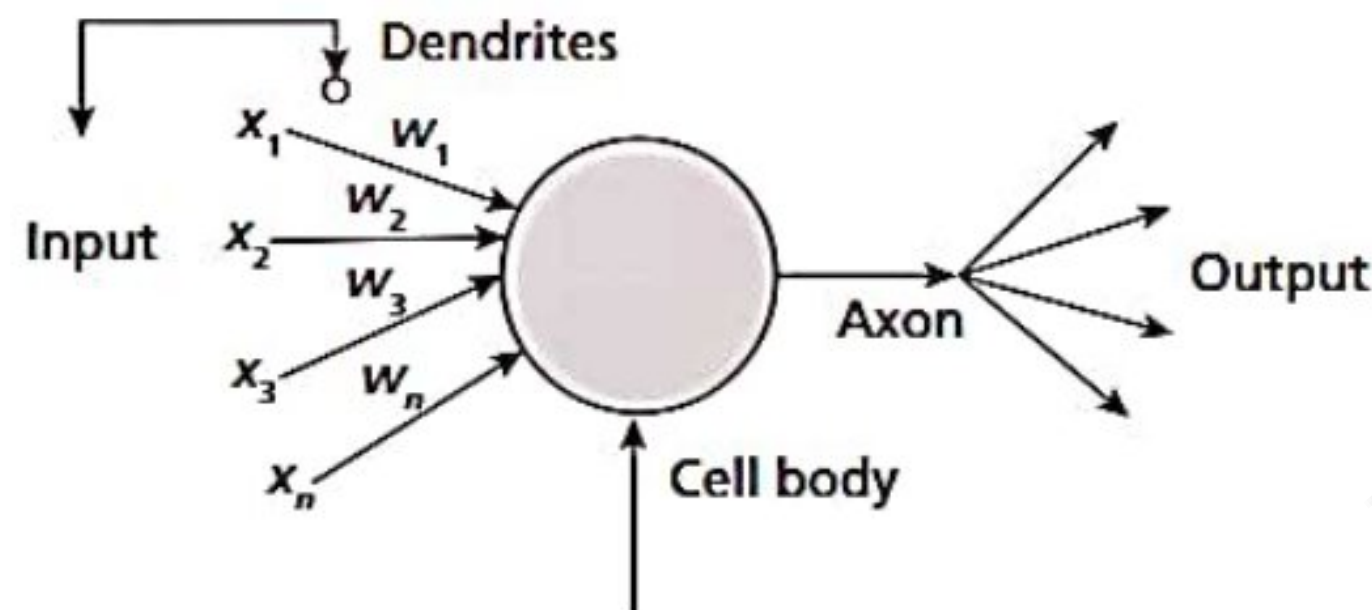


Figure 10.2: An Artificial Neuron

10.3.1 Simple Model of an Artificial Neuron

The first mathematical model of a biological neuron was designed by McCulloch & Pitts in 1943. It includes two steps:

1. It receives weighted inputs from other neurons
2. It operates with a threshold function or activation function

The received inputs are computed as a weighted sum which is given to the activation function and if the sum exceeds the threshold value the neuron gets fired. The mathematical model of a neuron is shown in Figure 10.3.

The neuron is the basic processing unit that receives a set of inputs $x_1, x_2, \dots, x_n$ and their associated weights $w_1, w_2, \dots, w_n$. The Summation function 'Net-sum' Eq. (10.1) computes the weighted sum of the inputs received by the neuron.

$$\text{Net-sum} = \sum_{i=1}^n x_i w_i \quad (10.1)$$

The activation function is a binary step function which outputs a value 1 if the Net-sum is above the threshold value θ , and a 0 if the Net-sum is below the threshold value θ . Therefore, the activation function is applied to Net-sum as shown in Eq. (10.2).

$$f(x) = \text{Activation function (Net - sum)} \quad (10.2)$$

$$\text{Then, output of a neuron } Y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \quad (10.3)$$

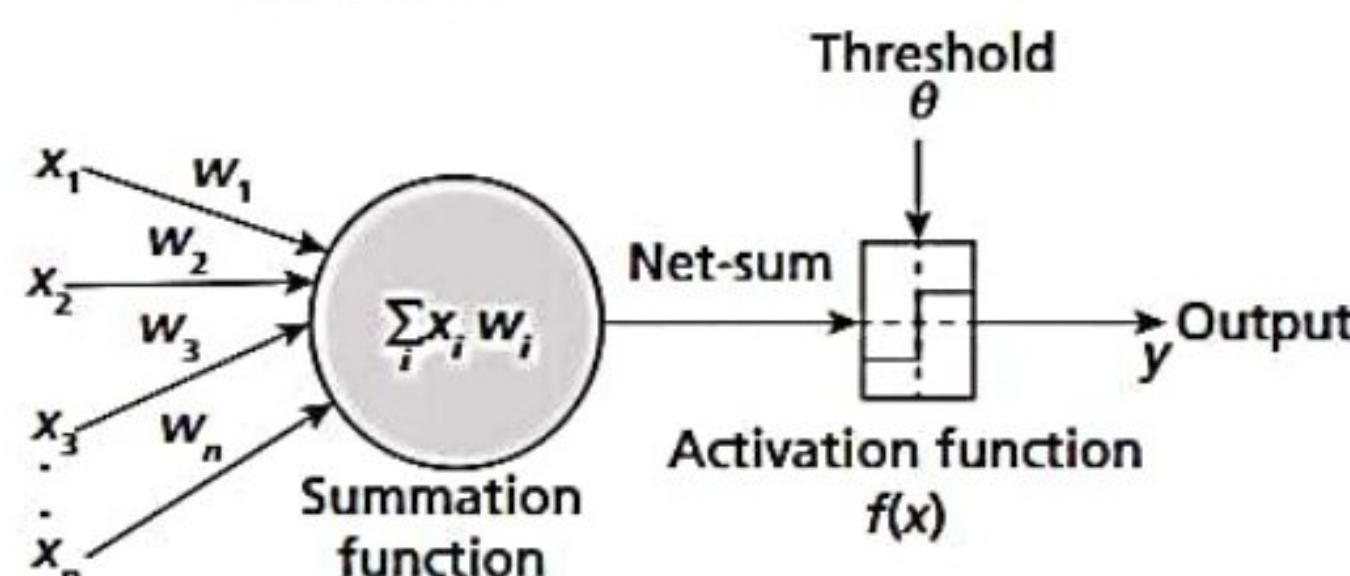


Figure 10.3: McCulloch & Pitts Neuron Mathematical Model

McCulloch & Pitts Neuron model can represent only a few Boolean functions. A Boolean function has binary inputs and provides a binary output. For example, an AND Boolean function neuron would fire when all the inputs are 1, whereas an OR Boolean function neuron would fire even when one input is 1. Moreover, the weight and threshold values are fixed in this mathematical model.

10.3.2 Artificial Neural Network Structure

Artificial Neural Network (ANN) imitates a human brain which inhibits some intelligence. It has a network structure represented as a directed graph with a set of neuron nodes and connection links or edges connecting the nodes as shown in Figure 10.4. The nodes in the graph are arrayed in a layered manner and can process information in parallel. The network given in the figure has three layers called input layer, hidden layer and output layer. The input layer receives the input information ($x_1, x_2, \dots, x_n$) and passes it to the nodes in the hidden layer. The edges connecting the nodes from the input layer to the hidden layer are associated with synaptic weights called as connection weights. These computing nodes or neurons perform some computations based on the input information ($x_1, x_2, \dots, x_n$) received and if the weighted sum of the inputs to a neuron is above the threshold or the activation level of the neuron, then the neuron fires. Each neuron employs an activation function that determines the output of the neuron. The neuron transforms linearly the input signals by computing the sum of the product of input signals and weights and adds biases to it. Then, the activation function maps the weighted input sum to a non-linear output value. The node in the output layer gives the output as a single value.

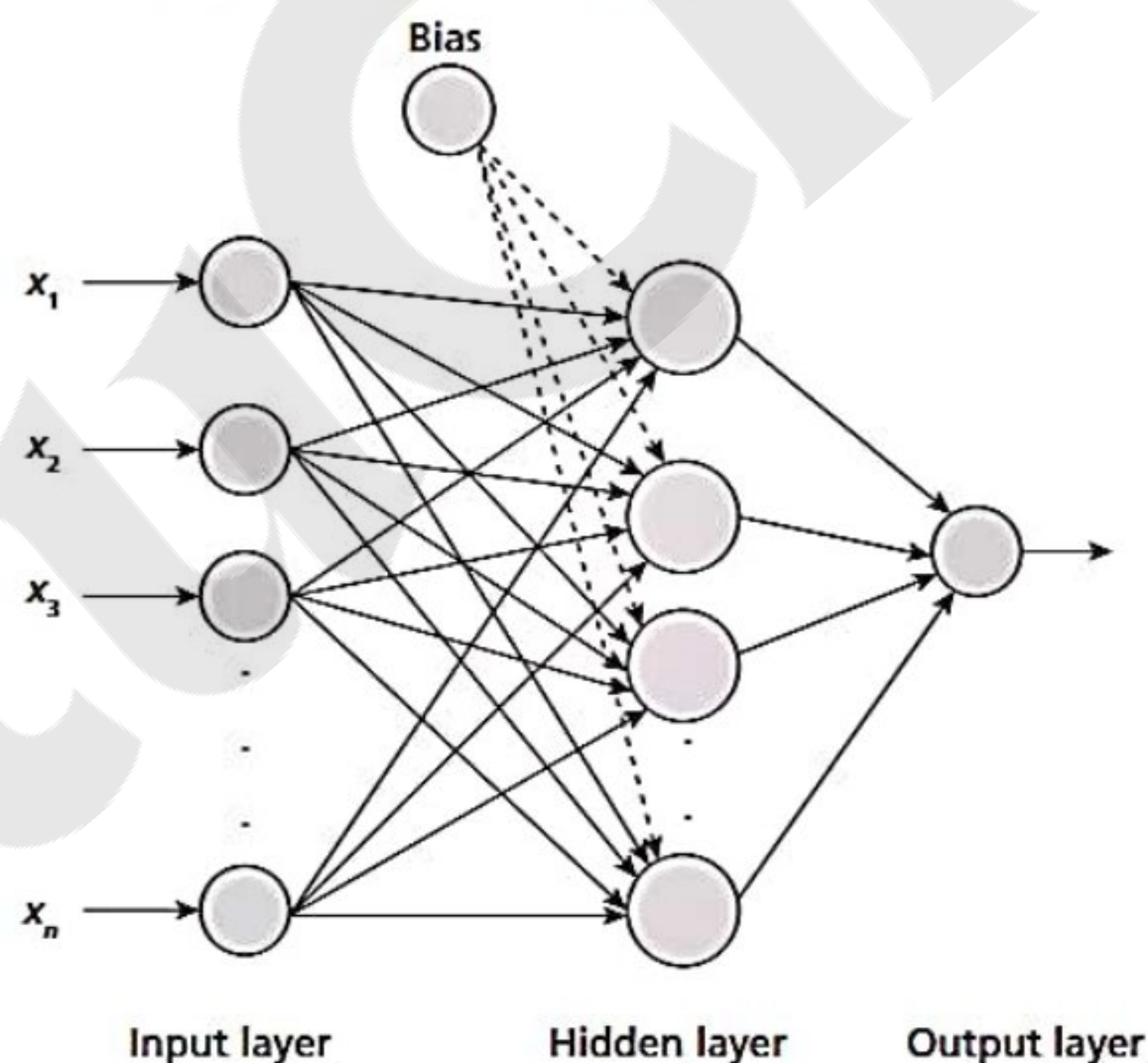


Figure 10.4: Artificial Neural Network Structure

10.3.3 Activation Functions

Activation functions are mathematical functions associated with each neuron in the neural network that map input signals to output signals. It decides whether to fire a neuron or not based on the input signals the neuron receives. These functions normalize the output value of each neuron either between 0 and 1 or between -1 and +1. Typical activation functions can be linear or non-linear.

Linear functions are useful when the input values can be classified into any one of the two groups and are generally used in binary perceptrons. Non-linear functions, on the other hand, are continuous functions that map the input in the range of (0, 1) or (-1, 1), etc. These functions are useful in learning high-dimensional data or complex data such as audio, video and images.

Below are some of the activation functions used in ANNs:

1. Identity Function or Linear Function

$$f(x) = x \quad \forall x \quad (10.4)$$

The value of $f(x)$ increases linearly or proportionally with the value of x . This function is useful when we do not want to apply any threshold. The output would be just the weighted sum of input values. The output value ranges between $-\infty$ and $+\infty$.

2. Binary Step Function

$$f(x) = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \quad (10.5)$$

The output value is binary, i.e., 0 or 1 based on the threshold value θ . If value of $f(x)$ is greater than or equal to θ , it outputs 1 or else it outputs 0.

3. Bipolar Step Function

$$f(x) = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ -1 & \text{if } f(x) < \theta \end{cases} \quad (10.6)$$

The output value is bipolar, i.e., +1 or -1 based on the threshold value θ . If value of $f(x)$ is greater than or equal to θ , it outputs +1 or else it outputs -1.

4. Sigmoidal Function or Logistic Function

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (10.7)$$

It is a widely used non-linear activation function which produces an S-shaped curve and the output values are in the range of 0 and 1. It has a vanishing gradient problem, i.e., no change in the prediction for very low input values and very high input values.

5. Bipolar Sigmoid Function

$$\sigma(x) = \frac{1 - e^{-x}}{1 + e^{-x}} \quad (10.8)$$

It outputs values between -1 and +1.

6. Ramp Functions

$$f(x) = \begin{cases} 1 & \text{if } x > 1 \\ x & \text{if } 0 \leq x \leq 1 \\ 0 & \text{if } x < 0 \end{cases} \quad (10.9)$$

It is a linear function whose upper and lower limits are fixed.

7. Tanh – Hyperbolic Tangent Function

The Tanh function is a scaled version of the sigmoid function which is also non-linear. It also suffers from the vanishing gradient problem. The output values range between -1 and 1.

$$\tanh(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (10.10)$$

8. ReLu – Rectified Linear Unit Function

This activation function is a typical function generally used in deep learning neural network models in the hidden layers. It avoids or reduces the vanishing gradient problem. This function outputs a value of 0 for negative input values and works like a linear function if the input values are positive.

$$r(x) = \max(0, x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \quad (10.11)$$

9. Softmax Function

This is a non-linear function used in the output layer that can handle multiple classes. It calculates the probability of each target class which ranges between 0 and 1. The probability of the input belonging to a particular class is computed by dividing the exponential of the given input value by the sum of the exponential values of all the inputs.

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=0}^k e^{x_j}} \text{ where } i = 0 \dots k \quad (10.12)$$

10.4 PERCEPTRON AND LEARNING THEORY

The first neural network model 'Perceptron', designed by Frank Rosenblatt in 1958, is a linear binary classifier used for supervised learning. He modified the McCulloch & Pitts Neuron model by combining two concepts, McCulloch-Pitts model of an artificial neuron and Hebbian learning rule of adjusting weights. He introduced variable weight values and an extra input that represents *bias* to this model. He proposed that artificial neurons could actually learn weights and thresholds from data and came up with a supervised learning algorithm that enabled the artificial neurons to learn the correct weights from training data by itself. The perceptron model (shown in Figure 10.5) consists of 4 steps:

1. Inputs from other neurons
2. Weights and bias
3. Net sum
4. Activation function

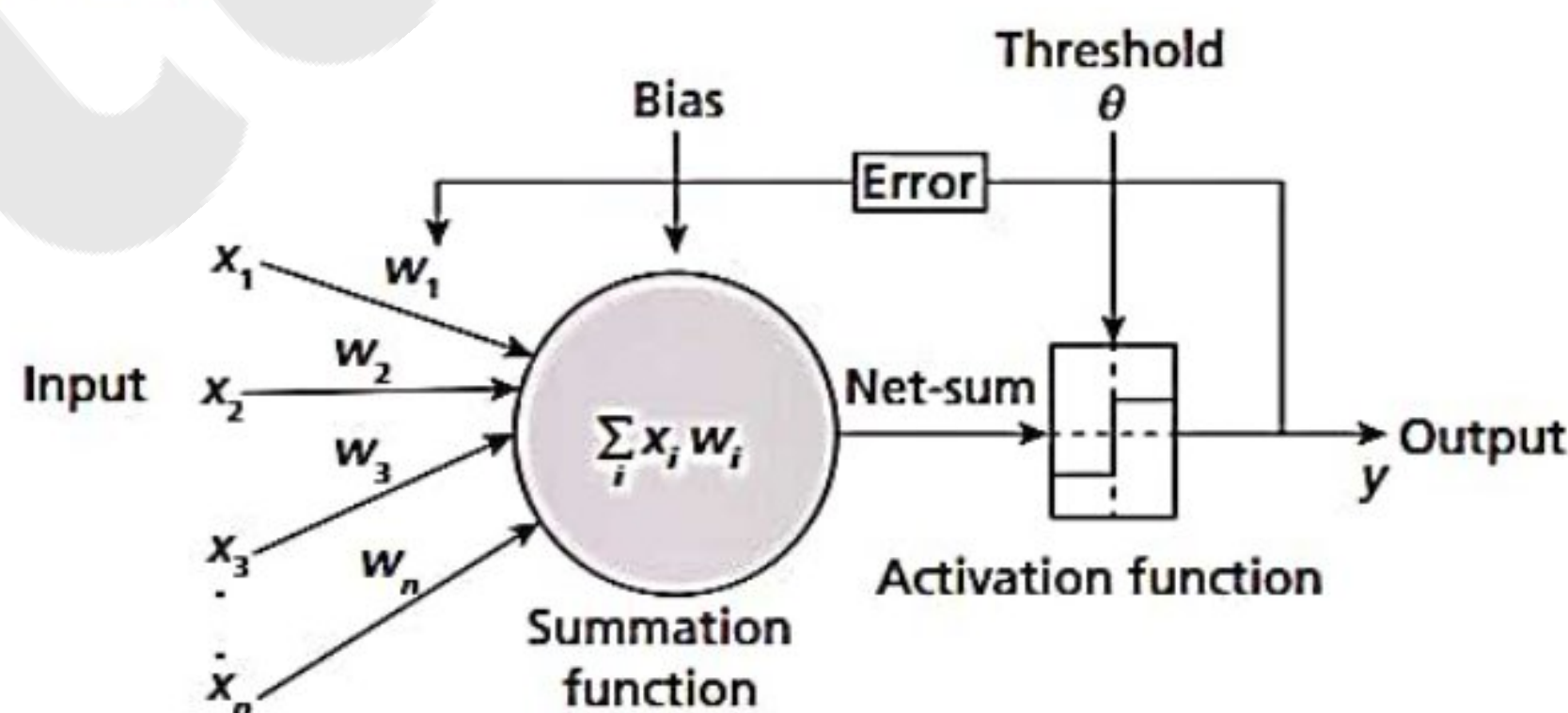


Figure 10.5: Perceptron Model

Thus, the modified neuron model receives a set of inputs $x_1, x_2, \dots, x_n$, their associated weights $w_1, w_2, \dots, w_n$ and a bias. The summation function 'Net-sum' Eq. (10.13) computes the weighted sum of the inputs received by the neuron.

$$\text{Net-sum} = \sum_{i=1}^n x_i w_i \quad (10.13)$$

After computing the 'Net-sum', bias value is added to it and inserted in the activation function as shown below:

$$f(x) = \text{Activation function} (\text{Net-sum} + \text{bias}) \quad (10.14)$$

The activation function is a binary step function which outputs a value 1 if $f(x)$ is above the threshold value θ , and a 0 if $f(x)$ is below the threshold value θ . Then, output of a neuron:

$$Y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \quad (10.15)$$

Before learning how a neural network works, let us learn about how a perceptron model works.

Algorithm 10.1: Perceptron Algorithm

Set initial weights $w_1, w_2, \dots, w_n$ and bias θ to a random value in the range $[-0.5, 0.5]$.

For each Epoch,

1. Compute the weighted sum by multiplying the inputs with the weights and add the products.
2. Apply the activation function on the weighted sum:

$$Y = \text{Step} ((x_1 w_1 + x_2 w_2) - \theta)$$

3. If the sum is above the threshold value, output the value as positive else output the value as negative.
4. Calculate the error by subtracting the estimated output $Y_{\text{estimated}}$ from the desired output Y_{desired} :

$$\text{error } e(t) = Y_{\text{desired}} - Y_{\text{estimated}}$$

[If error $e(t)$ is positive, increase the perceptron output Y and if it is negative, decrease the perceptron output Y .]

5. Update the weights if there is an error:

$$\Delta w_i = \alpha \times e(t) \times x_i$$

$$w_i = w_i + \Delta w_i$$

where, x_i is the input value, $e(t)$ is the error at step t , α is the learning rate and Δw_i is the difference in weight that has to be added to w_i .

Example 10.1: Consider a perceptron to represent the Boolean function AND with the initial weights $w_1 = 0.3$, $w_2 = -0.2$, learning rate $\alpha = 0.2$ and bias $\theta = 0.4$ as shown in Figure 10.6. The activation function used here is the Step function $f(x)$ which gives the output value as binary, i.e., 0 or 1. If value of $f(x)$ is greater than or equal to 0, it outputs 1 or else it outputs 0. Design a perceptron that performs the Boolean function AND and update the weights until the Boolean function gives the desired output.

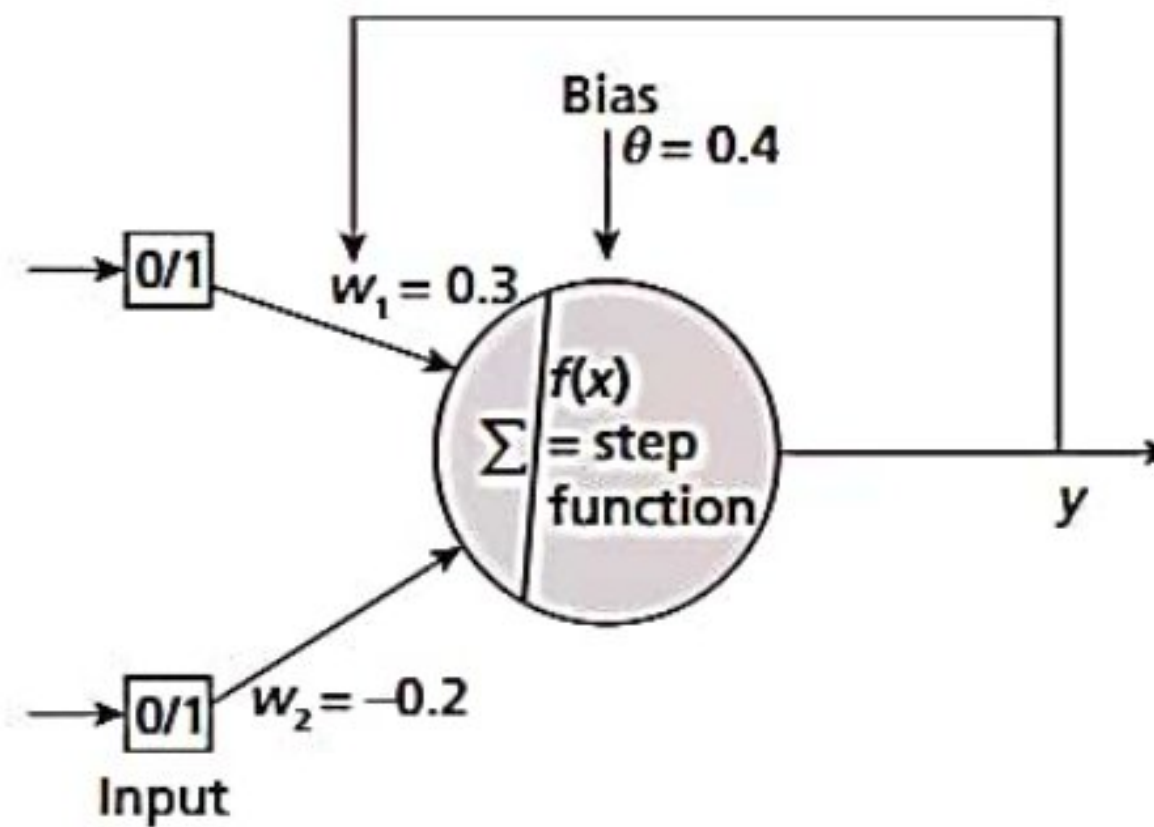


Figure 10.6: Perceptron for Boolean Function AND

Solution: Desired output for Boolean function AND is shown in Table 10.1.

Table 10.1: AND Truth Table

x_1	x_2	Y_{des}
0	0	0
0	1	0
1	0	0
1	1	1

For each Epoch, weighted sum is calculated and the activation function is applied to compute the estimated output Y_{est} . Then, Y_{est} is compared with Y_{des} to find the error. If there is an error, the weights are updated.

Tables 10.2 to 10.5 show how the weights are updated in the four Epochs.

Table 10.2: Epoch 1

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
1	0	0	0	Step $((0 \times 0.3 + 0 \times -0.2) - 0.4) = 0$	0	0.3	-0.2	No change
	0	1	0	Step $((0 \times 0.3 + 1 \times -0.2) - 0.4) = 0$	0	0.3	-0.2	No change
	1	0	0	Step $((1 \times 0.3 + 0 \times -0.2) - 0.4) = 0$	0	0.3	-0.2	No change
	1	1	1	Step $((1 \times 0.3 + 1 \times -0.2) - 0.4) = 0$	1	0.5	0	Change

For input (1, 1) the weights are updated as follows:

$$\Delta w_1 = \alpha \times e(t) \times x_1 = 0.2 \times 1 \times 1 = 0.2$$

$$w_1 = w_1 + \Delta w_1 = 0.3 + \Delta w_1 = 0.3 + 0.2 = 0.5$$

$$\Delta w_2 = \alpha \times e(t) \times x_2 = 0.2 \times 1 \times 1 = 0.2$$

$$w_2 = w_2 + \Delta w_2 = -0.2 + \Delta w_2 = -0.2 + 0.2 = 0$$

Table 10.3: Epoch 2

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
2	0	0	0	Step $((0 \times 0.5 + 0 \times 0) - 0.4) = 0$	0	0.5	0	No change
	0	1	0	Step $((0 \times 0.5 + 1 \times 0) - 0.4) = 0$	0	0.5	0	No change
	1	0	0	Step $((1 \times 0.5 + 0 \times 0) - 0.4) = 1$	-1	0.3	0	Change
	1	1	1	Step $((1 \times 0.3 + 1 \times 0) - 0.4) = 0$	1	0.5	0.2	Change

For input (1, 0) the weights are updated as follows:

$$\Delta w_1 = \alpha \times e(t) \times x_1 = 0.2 \times -1 \times 1 = -0.2$$

$$w_1 = w_1 + \Delta w_1 = 0.5 + \Delta w_1 = 0.5 - 0.2 = 0.3$$

$$\Delta w_2 = \alpha \times e(t) \times x_2 = 0.2 \times -1 \times 0 = 0$$

$$w_2 = w_2 + \Delta w_2 = 0 + \Delta w_2 = 0 + 0 = 0$$

For input (1, 1), the weights are updated as follows:

$$\Delta w_1 = \alpha \times e(t) \times x_1 = 0.2 \times 1 \times 1 = 0.2$$

$$w_1 = w_1 + \Delta w_1 = 0.3 + \Delta w_1 = 0.3 + 0.2 = 0.5$$

$$\Delta w_2 = \alpha \times e(t) \times x_2 = 0.2 \times 1 \times 1 = 0.2$$

$$w_2 = w_2 + \Delta w_2 = 0 + \Delta w_2 = 0 + 0.2 = 0.2$$

Table 10.4: Epoch 3

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
3	0	0	0	Step $((0 \times 0.5 + 0 \times 0.2) - 0.4) = 0$	0	0.5	0.2	No change
	0	1	0	Step $((0 \times 0.5 + 1 \times 0.2) - 0.4) = 0$	0	0.5	0.2	No change
	1	0	0	Step $((1 \times 0.5 + 0 \times 0.2) - 0.4) = 1$	-1	0.3	0.2	Change
	1	1	1	Step $((1 \times 0.3 + 1 \times 0.2) - 0.4) = 1$	0	0.3	0.2	No change

For input (1, 0) the weights are updated as follows:

$$\Delta w_1 = \alpha \times e(t) \times x_1 = 0.2 \times -1 \times 1 = -0.2$$

$$w_1 = w_1 + \Delta w_1 = 0.5 + \Delta w_1 = 0.5 - 0.2 = 0.3$$

$$\Delta w_2 = \alpha \times e(t) \times x_2 = 0.2 \times -1 \times 0 = 0$$

$$w_2 = w_2 + \Delta w_2 = 0 + \Delta w_2 = 0.2 + 0 = 0.2$$

Table 10.5: Epoch 4

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
4	0	0	0	Step $((0 \times 0.3 + 0 \times 0.2) - 0.4) = 0$	0	0.3	0.2	No change
	0	1	0	Step $((0 \times 0.3 + 1 \times 0.2) - 0.4) = 0$	0	0.3	0.2	No change
	1	0	0	Step $((1 \times 0.3 + 0 \times 0.2) - 0.4) = 0$	0	0.3	0.2	No change
	1	1	1	Step $((1 \times 0.3 + 1 \times 0.2) - 0.4) = 1$	0	0.3	0.2	No change

It is observed that with 4 Epochs, the perceptron learns and the weights are updated to 0.3 and 0.2 with which the perceptron gives the desired output of a Boolean AND function.

10.4.1 XOR Problem

A perceptron model can solve all Boolean functions which are linearly separable. However, the XOR problem was identified in 1969 by Minsky and Papert. An XOR function returns a 1, if the two inputs are not equal and a 0 if they are equal. Following is the truth table of an XOR function shown in Table 10.6.

Table 10.6: XOR Truth Table

x_1	x_2	Y
0	0	1
0	1	0
1	0	0
1	1	1

Since XOR is not linearly separable, the single layer perceptron failed to classify and hence this problem led to the evolution of a multi-layer perceptron. Initially, the Multi-Layer Perceptron (MLP) was not a success due to the lack of an appropriate learning algorithm. In 1974, Werbos introduced a back propagation algorithm for the three-layered perceptron network and in 1986, a general back propagation algorithm for a multi-layered perceptron was introduced by Rumelhart and McClelland. Then, again ANN and Deep Neural Networks became a success, solving many complex problems in the current era. Thus, MLP that emerged could solve any non-linear separable problem which is explained later in this chapter.

10.4.2 Delta Learning Rule and Gradient Descent

Generally, learning in neural networks is performed by adjusting the network weights in order to minimize the difference between the desired and estimated outputs. This delta difference is measured as an error function or also called as cost function. The cost function, being linear and continuous, is differentiable. This way of learning called as delta rule (also known as Widrow-Hoff rule or Adaline rule) is a type of back propagation applied for training the network. The training error of a hypothesis is half the squared difference between the desired target output and actual output and is given as follows:

$$\text{Training Error} = \frac{1}{2} \sum_{d \in T} (O_{\text{Desired}} - O_{\text{Estimated}})^2 \quad (10.16)$$

where, T is the training dataset, O_{Desired} and $O_{\text{Estimated}}$ are the desired target output and estimated actual output, respectively, for a training instance d .

The principle of gradient descent is an optimization approach which is used to minimize the cost function by converging to a local minimal point moving in the negative direction of the gradient and each step size during movement is determined by the learning rate and the slope of the gradient.

Gradient descent learning is the foundation of back propagation algorithm used in MLP. Before we study about an MLP, let us first understand the different types of neural networks that differ in their structure, activation function and learning mechanism.

10.5 TYPES OF ARTIFICIAL NEURAL NETWORKS

ANNs consist of multiple neurons arranged in layers. There are different types of ANNs that differ by the network structure, activation function involved and the learning rules used. In an ANN, there are three layers called input layer, hidden layer and output layer. Any general ANN would consist of one input layer, one output layer and zero or more hidden layers.

10.5.1 Feed Forward Neural Network

This is the simplest neural network that consists of neurons which are arranged in layers and the information is propagated only in the forward direction. This model may or may not contain a hidden layer and there is no back propagation. Based on the number of hidden layers they are further classified into single-layered and multi-layered feed forward networks. These ANNs are simple to design and easy to maintain. They are fast but cannot be used for complex learning. They are used for simple classification and simple image processing, etc. The model of a Feed Forward Neural Network is shown in Figure 10.7.

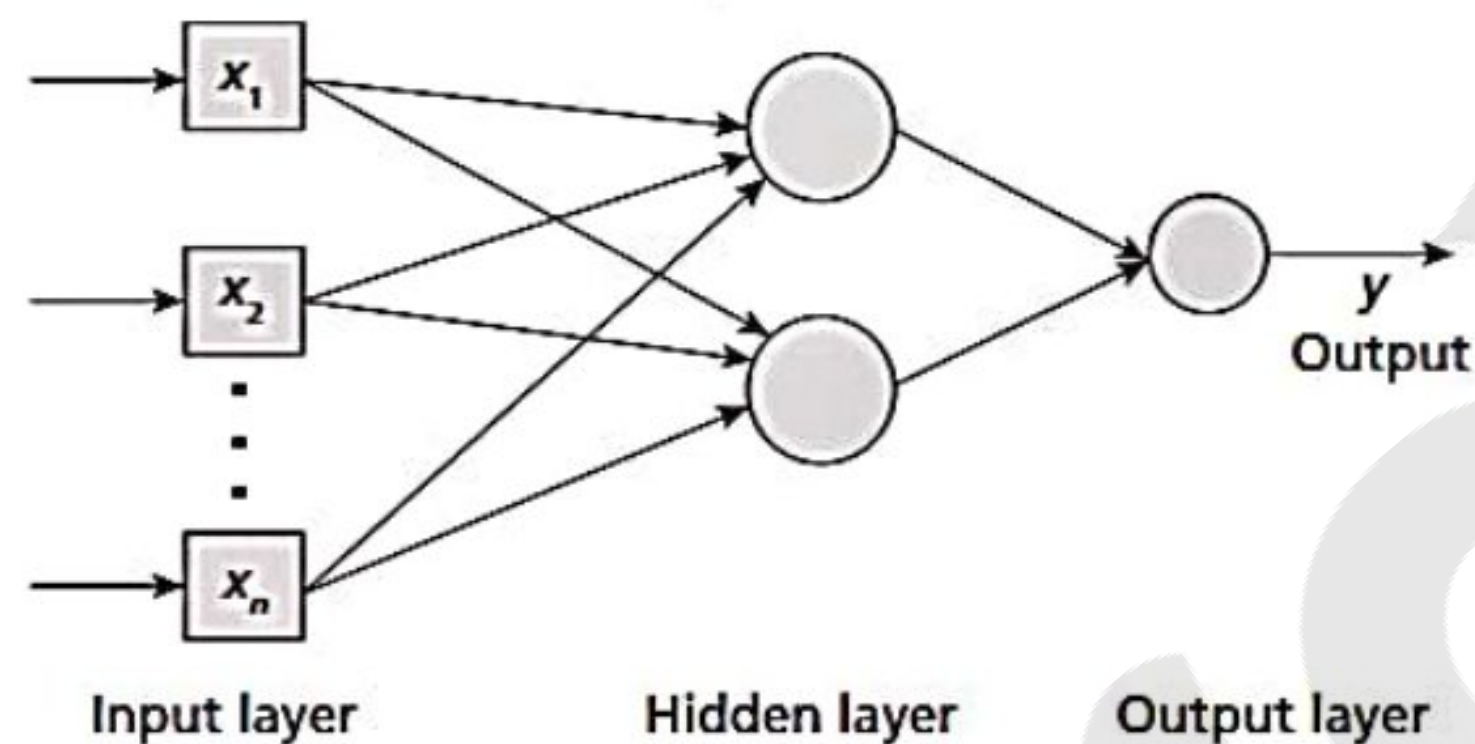


Figure 10.7: Model of a Feed Forward Neural Network

10.5.2 Fully Connected Neural Network

Fully connected neural networks are the ones in which all the neurons in a layer are connected to all other neurons in the next layer. The model of a fully connected neural network is shown in Figure 10.8.

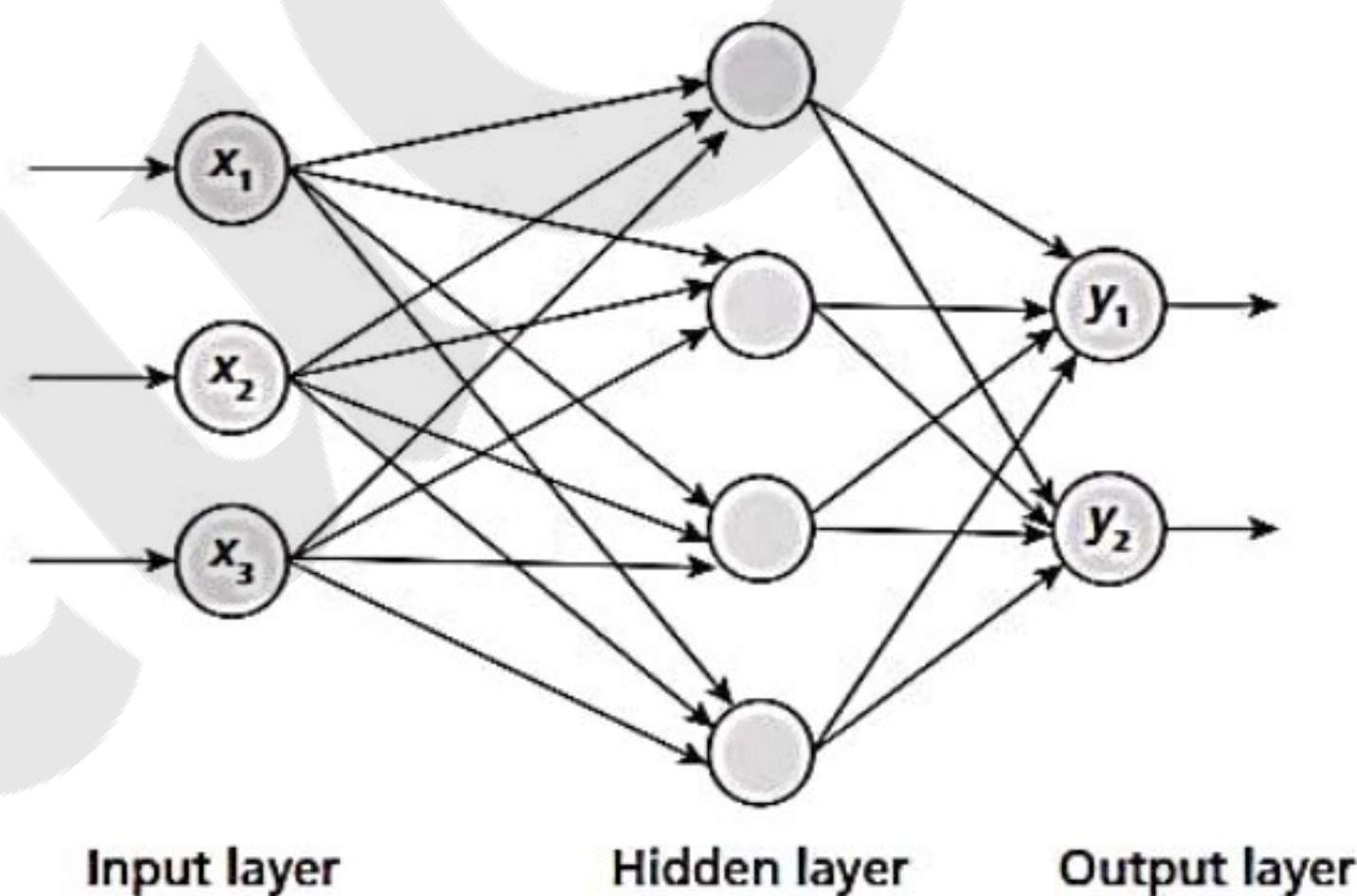


Figure 10.8: Model of a Fully Connected Neural Network

10.5.3 Multi-Layer Perceptron (MLP)

This ANN consists of multiple layers with one input layer, one output layer and one or more hidden layers. Every neuron in a layer is connected to all neurons in the next layer and thus they are fully connected. The information flows in both the directions. In the forward direction, the inputs are multiplied by weights of neurons and forwarded to the activation function of the

neuron and output is passed to the next layer. If the output is incorrect, then in the backward direction, error is back propagated to adjust the weights and biases to get correct output. Thus, the network learns with the training data. This type of ANN is used in deep learning for complex classification, speech recognition, medical diagnosis, forecasting, etc. They are comparatively complex and slow. The model of an MLP is shown in Figure 10.9.

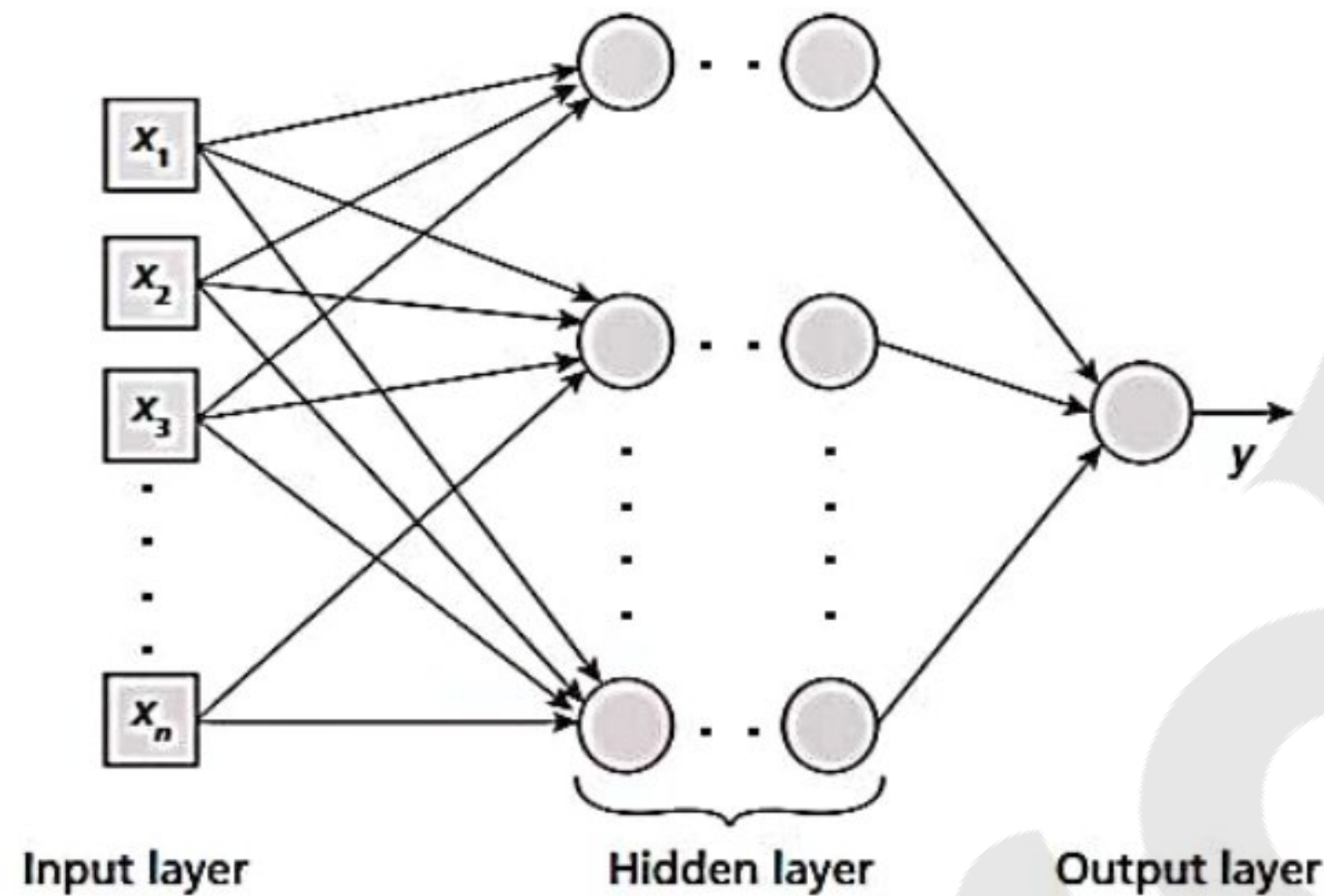


Figure 10.9: Model of a Multi-Layer Perceptron

10.5.4 Feedback Neural Network

Feedback neural networks have feedback connections between neurons that allow information flow in both directions in the network. The output signals can be sent back to the neurons in the same layer or to the neurons in the preceding layers. Hence, this network is more dynamic during training. The model of a feedback neural network is shown in Figure 10.10.

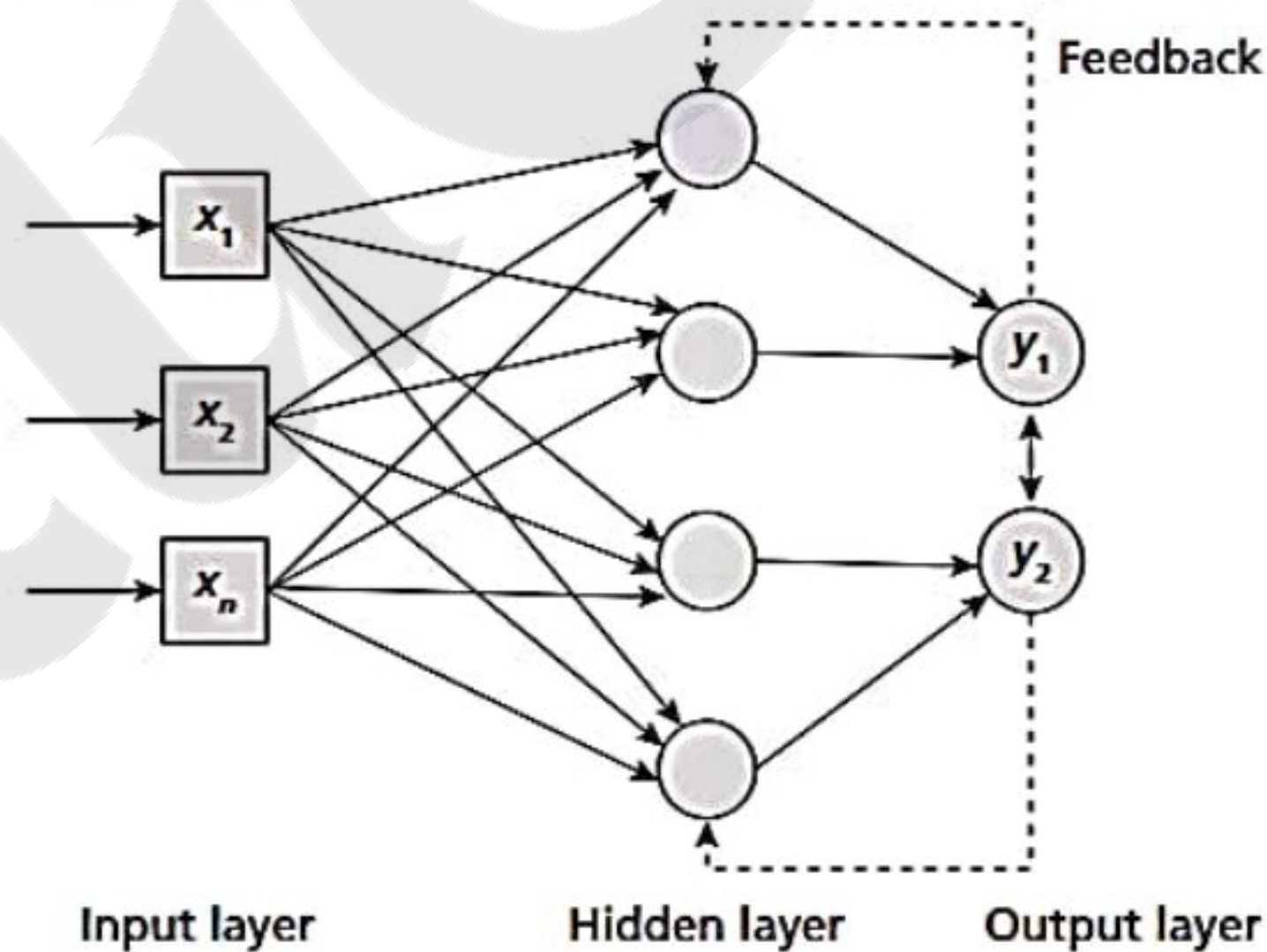


Figure 10.10: Model of a Feedback Neural Network

10.9 POPULAR APPLICATIONS OF ARTIFICIAL NEURAL NETWORKS

ANN learning mechanisms are used in many complex applications that involve modelling of non-linear processes. ANN is a useful model that can handle even noisy and incomplete data. They are used to model complex patterns, recognize patterns and solve prediction problems like humans in many areas such as:

1. Real-time applications: Face recognition, emotion detection, self-driving cars, navigation systems, routing systems, target tracking, vehicle scheduling, etc.
2. Business applications: Stock trading, sales forecasting, customer behaviour modelling, Market research and analysis, etc.
3. Banking and Finance: Credit and loan forecasting, fraud and risk evaluation, currency price prediction, real-estate appraisal, etc.
4. Education: Adaptive learning software, student performance modelling, etc.
5. Healthcare: Medical diagnosis or mapping symptoms to a medical case, image interpretation and pattern recognition, drug discovery, etc.
6. Other Engineering Applications: Robotics, aerospace, electronics, manufacturing, communications, chemical analysis, food research, etc.

10.10 ADVANTAGES AND DISADVANTAGES OF ANN

Advantages of ANN

1. ANN can solve complex problems involving non-linear processes.
2. ANNs can learn and recognize complex patterns and solve problems as humans solve a problem.
3. ANNs have a parallel processing capability and can predict in less time.
4. They have an ability to work with inadequate knowledge. It can even handle incomplete and noisy data.
5. They can scale well to larger data sets and outperforms other learning mechanisms.

Limitations of ANN

1. An ANN requires processors with parallel processing capability to train the network running for many epochs. The function of each node requires a CPU capability which is difficult for very large networks with a large amount of data.
2. They work like a 'black box' and it is exceedingly difficult to understand their working in inner layers. Moreover, it is hard to understand the relationship between the representations learned at each layer.

3. The modelling with ANN is also extremely complicated and the development takes a much longer time.
4. Generally, neural networks require more data than traditional machine learning algorithms, and they do not perform well on small datasets.
5. They are also more computationally expensive than traditional learning techniques.

10.11 CHALLENGES OF ARTIFICIAL NEURAL NETWORKS

The major challenges while modelling a real-time application with ANNs are:

1. Training a neural network is the most challenging part of using this technique. Overfitting or underfitting issues may arise if datasets used for training are not correct. It is also hard to generalize to the real-world data when trained with some simulated data. Moreover, neural network models normally need a lot of training data to be robust and are usable for a real-time application.
2. Finding the weight and bias parameters for neural networks is also hard and it is difficult to calculate an optimal model.